Les protocoles applicatifs

Différence entre application Réseau et protocoles de la couche Application

Application réseau	Protocole de la couche Application
Service / logiciel utilisé par l'utilisateur	**Ensemble de règles de communication**
Réalise une fonction pour l'utilisateur	Définit **comment les applications communiquent**
Exemples : navigateur Web, client de messagerie, client FTP	Exemples : **HTTP, SMTP, FTP, DNS, MQTT, CoAP**
Peut utiliser un ou plusieurs protocoles	Est utilisé par une application pour échanger des données

Exemple : le Web
Navigateur Web → *application réseau*
⬇ utilise
HTTP / HTTPS → *protocole de la couche Application*
⬇ transporte les données via
TCP/IP → couches inférieures

Les protocoles du Web

HTTP, HTTPS, DNS

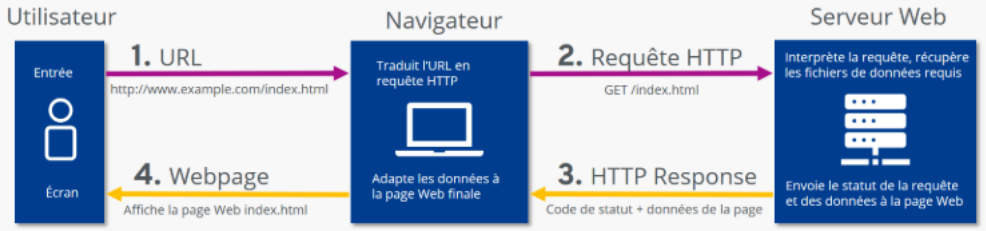
HTTP/HTTPS

- HTTP : HyperText Tranfert Protocol
- **protocole réseau** utilisé par les **navigateurs Web** (Firefox, Internet Explorer...) et les **serveurs Web** (Apache, Nginx, Microsoft IIS...) pour communiquer entre eux.
- Le **protocole HTTP** permet également de **transmettre des données au serveur**, par l'intermédiaire de **formulaires** Internet.
- S'appuie les protocoles plus bas-niveau **TCP** et **IP** (voir le modèle *Internet* ci-dessous).
- Application Client/Serveur
- HTTPS (HTTP Secure): HTTP protégé par TLS (Transport Layer Security)

Constituants d'une page web

- Une page Web est cnstitué de différents objets.
- Un objet est un fichier:
  - Fichier HTML
  - Une image GIF ou JPEG
  - Une applette Java
  - Un clip audio
  - …..
- Les objets sont identifiés par des URL (Uniform Resource Locator)
- L'URL est composé de deux éléments:
  - Le nom du serveur hébergeant l'objet
  - Le chemin de l'objet

Principe Fondamental de Fonctionnement



- Ressources échangées:
  - pages HTML
  - images
  - feuilles CSS
  - fichiers JavaScript
  - vidéos
  - données JSON
  - réponses d'API

1) Le client établit une connexion TCP avec le serveur Web (habituellement sur le port 80) et émet une demande de document en se servant de l'instruction GET du protocole HTTP.
2) Le serveur reçoit la demande, l'examine et en cas de succès retourne le document demandé.
3) Le serveur clôture ensuite la session.

Remarque: Le serveur HTTP ne conserve aucune information sur ses clients, on parle de protocole sans mémoire (stateless protocol)

HTTP
Fonctionnement

- Connexion
- demande (GET) d 'un document
- renvoi du document (status=200) ou d 'une erreur
- déconnexion

Format d'une requête HTTP

Une requête contient notamment :

- une méthode
- une URI
- des en-têtes
- éventuellement un corps

<Méthode> <URI> HTTP/<Version>
[<Champ d 'entête>: <Valeur>]
[<tab><Suite Valeur si >1024>]
ligne blanche
[corps de la requête pour la méthode POST]

- http://www.univ-usto.dz/index.htm
- Requête HTTP:
  - GET /index.html HTTP/1.1
  - Host: www.example.com
  - User-Agent: Mozilla/5.0
  - Accept: text/html
  - Connection: keep-alive

Les méthodes HTTP

Méthode	Utilisation
GET	récupérer une ressource
POST	soumettre/créer une ressource
PUT	remplacer une ressource
PATCH	modifier partiellement
DELETE	supprimer
HEAD	obtenir les en-têtes uniquement
OPTIONS	connaître les possibilités du serveur

Les méthodes HTTP

POST — Créer une ressource

```
POST /etudiants HTTP/1.1
Host: universite.dz
Content-Type: application/json

{
  "nom": "Amine",
  "prenom": "Karim"
}
```

PUT — Remplacer une ressource

```
PUT /etudiants/15 HTTP/1.1
Host: universite.dz
Content-Type: application/json

{
  "nom": "Amine",
  "prenom": "Karim",
  "email": "amine@universite.dz"
}
```

Les méthodes HTTP

PATCH — Modifier partiellement une ressource

```
PATCH /etudiants/15 HTTP/1.1
Host: universite.dz
Content-Type: application/json

{
  "email": "nouveau@universite.dz"
}
```

DELETE — Supprimer une ressource

```
DELETE /etudiants/15 HTTP/1.1
Host: universite.dz
```

HEAD — Obtenir les en-têtes uniquement

```
HEAD /etudiants/15 HTTP/1.1
Host: universite.dz
```

OPTIONS — Connaître les méthodes disponibles

```
OPTIONS /etudiants HTTP/1.1
Host: universite.dz
```

Exemple de réponse :

```
HTTP/1.1 200 OK
Allow: GET, POST, OPTIONS
```

L'en-tête Allow indique ici les méthodes prises en charge pour cette ressource

Format d'une réponse HTTP

Trois parties

1. **Status line**
2. **Headers**
3. **Body**

```
HTTP/<Version> <Status> <Commentaire Status>
Content-Type: <Type MIME du contenu>
[< Champ d 'entête >: <Valeur>]
[<tab><Suite Valeur si >1024>]
Ligne blanche
Document
```

```
HTTP/1.0 200 OK
Date: Wed, 02Feb97 23:04:12 GMT
```

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1250
Server: Apache

<html>
   ...
</html>
```

Les codes de statut HTTP

- **1xx — Information**
  - 100 Continue
  - Le serveur indique que la requête a été reçue et que le traitement se poursuit.
- **2xx — Succès**
  - 200 OK
  - 201 Created (réponse pour les requêtes de creation de ressources)
  - 204 No Content (requête réussie, mais je n'ai rien à te renvoyer. En cas de modification ou de suppression par exemple)
- **3xx — Redirection**
  - 301 Moved Permanently (La ressource a changé d'adresse de manière permanente. indique la nouvelle adresse dans l'en-tête Location.)
  - 302 Found (redirection temporaire, va à l'adresse suivante pour le moment)
  - 304 Not Modified (La ressource n'a pas changé. Tu peux utiliser ta copie en cache)

4xx — Erreur client

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

5xx — Erreur serveur

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable

Les codes de statut HTTP

A. Codes de succès : 2xx

Ces codes indiquent un succès, mais pas nécessairement le même type de résultat

Code	Signification	Exemple concret
200 OK	Requête réussie.	Le navigateur reçoit le contenu d'une page Web.
201 Created	Une nouvelle ressource a été créée.	Un étudiant est ajouté à la base de données après une requête POST.
202 Accepted	La requête a été acceptée, mais son traitement n'est pas terminé.	Un serveur accepte une demande de génération d'un rapport qui sera préparé ultérieurement.
204 No Content	Requête réussie, sans contenu dans la réponse.	Une application modifie les informations d'un étudiant sans avoir besoin de recevoir des données en retour.

Les codes de statut HTTP
Codes de redirection : 3xx

Code	Signification	Exemple concret
301 Moved Permanently	Redirection permanente.	Un site a définitivement changé d'adresse.
302 Found	Redirection temporaire.	Un site redirige temporairement l'utilisateur vers une autre page.
304 Not Modified	La ressource n'a pas changé depuis la version en cache.	Le navigateur réutilise une copie locale d'une image au lieu de la télécharger à nouveau.

HTTP/1.1 301 Moved Permanently
Location: https://nouveau-site.dz/

L'en-tête **Location** indique la nouvelle adresse vers laquelle le client doit être redirigé.

Les codes de statut HTTP

Codes d'erreur côté client : 4xx

Code	Signification	Exemple concret
400 Bad Request	Requête mal formée ou invalide.	Une API reçoit des données JSON syntaxiquement incorrectes.
401 Unauthorized	Authentification nécessaire ou invalide.	Un utilisateur tente d'accéder à son compte sans fournir de justificatifs d'authentification valides.
403 Forbidden	Accès refusé.	Un étudiant authentifié tente d'accéder à une fonction réservée à l'administration.
404 Not Found	Ressource introuvable.	Un navigateur demande une page qui n'existe pas à l'adresse indiquée.
405 Method Not Allowed	Méthode HTTP non autorisée pour cette ressource.	Une API autorise GET sur /etudiants, mais pas DELETE.
408 Request Timeout	Le serveur interrompt l'attente d'une requête qui tarde trop.	Un client ne transmet pas sa requête dans le délai attendu par le serveur.
429 Too Many Requests	Trop de requêtes ont été envoyées pendant une période donnée.	Une API limite le nombre d'appels effectués par un utilisateur.

Les codes de statut HTTP

Codes d'erreur côté serveur : 5xx

Code	Signification	Exemple concret
`500 Internal Server Error`	Erreur interne inattendue.	Une exception non gérée se produit dans le programme du serveur.
`501 Not Implemented`	Fonctionnalité ou méthode non prise en charge par le serveur.	Un serveur ne sait pas traiter une méthode HTTP demandée.
`502 Bad Gateway`	Un serveur intermédiaire reçoit une réponse invalide d'un autre serveur.	Un proxy reçoit une réponse incorrecte du serveur applicatif.
`503 Service Unavailable`	Service temporairement indisponible.	Un serveur est surchargé ou en maintenance.
`504 Gateway Timeout`	Un serveur intermédiaire ne reçoit pas à temps la réponse d'un autre serveur.	Une passerelle attend trop longtemps la réponse du serveur applicatif.

Les headers HTTP (les en-têtes)

- Les headers transmettent des informations supplémentaires.
- Headers de requête
  - **Accept** = type MIME (**Multipurpose Internet Mail Extensions**.) visualisable par l 'agent
    - text/html → page HTML
    - text/plain → texte
    - image/jpeg → image JPEG
    - image/png → image PNG
    - application/json → données JSON
    - application/pdf → fichier PDF
    - video/mp4 → vidéo
  - **Accept-Language** : liste de langues (fr, en, …)
  - **Authorization** = type d 'autorisation (des informations d'authentification au serveur)
    - **BASIC nom:** mot de passe (en base64) donc transmis en clair !!!
    - **NB :** Préalablement le serveur a répondu un WWW-Authenticate
  - **Cookie** = cookie rétourné
  - **Host** = spécifie la machine et le port du serveur
    - un serveur peut héberger plusieurs serveurs
  - **Referer =** URL d 'origine
    - page à contenant l 'ancre à partir de laquelle le visualisateur a trouvé l 'URL.
  - **User-Agent** = modèle du visualisateur
  - **From** = adresse email de l 'utilisateur
    - rarement envoyé pour conserver l anonymat de l 'utilisateur
  - **If-Modified-Since** = condition de retrait
    - la page n 'est transférée que si elle a été modifiée depuis la date précisée. Utilisé par les caches
  - **If-Unmodified-Since** = condition de retrait
  - **If-Match** = condition de retrait
  - **If-None-Match** = condition de retrait
  - **If-Range** = condition de retrai
  - **Max-Forwards** = nombre max de proxy
  - **Proxy-Authorization** = identification
  - **Range** = zone du document à renvoyer
    - bytes=x-y (x=0 correspond au premier octet, y peut être omis pour spécifier jusqu 'à la fin)

Les headers HTTP (les en-têtes)
Headers de réponse

- **Accept-Range** = accepte ou refus d 'une requête par intervalle
- **Age** = ancienneté du document en secondes
- **Proxy-Authenticate** = système d 'authentification du proxy
- **Public** = liste de méthodes non standards gérées par le serveur (obsolète)
- **Retry-After** = date ou nombre de secondes pour un reessai en cas de code 503 (service unavailable)
- **Server** = modèle de HTTPD
- **Set-Cookie** = créer ou modifie un cookie sur le client
  - **Vary** = l 'entité possède plusieurs sources
  - **Warning** = informations supplémentaire du code d 'état
  - **Transformation applied** : le proxy a changé de Content-Type ou le Content-Encoding (obsolète)
- **WWW-Authenticate** = système d 'authentification pour l 'URI
- **Allow** = méthodes autorisées pour l 'URI
- **Content-Base** = URI de base pour la résolution des URL
- **Last-Modified** = date de dernière modification du doc.
  - Utilisé par les caches
- **Content-Length** = taille du document en octet
  - utilisé par le client pour gauger la progression des chargements

Les headers HTTP (les en-têtes)
Headers de réponse (suite)

- **Content-Encoding** = type encodage du document renvoyé
  - compress, x-gzip, x-zip
- **Content-Language** : le langage du document retourné
  - fr, en ...
- **Content-Location** : URI de l 'entité
  - quand l 'URI est à plusieurs endroit
- **Content-Range** : position du corps partiel dans l 'entité
  - bytes x-y/taille
- **Content-Transfert-Encoding** : transformation appliqué du corps de l 'entité
  - 7bit, binary, base64, quoted-printable
- **Content-Type** = type MIME du document renvoyé
  - utilisé par le client pour sélectionner le visualisateur (plugin)
- **ETag :** transformation appliqué du corps de l 'entité (Entity tag: code donné à une entité)
- **Expires** : date de péremption de l 'entité
- **Refresh** : 10
- **Refresh:** 10; Location=/newloc.htm

Exemple d'échange HTTP complet

- Requête
  - **GET /products/10 HTTP/1.1**
  - **Host: shop.example.com**
  - **Accept: application/json**
- Réponse
  - **HTTP/1.1 200 OK**
  - **Content-Type: application/json**
  - **{**
  - **"id": 10,**
  - **"name": "Laptop",**
  - **"price": 1200**
  - **}**

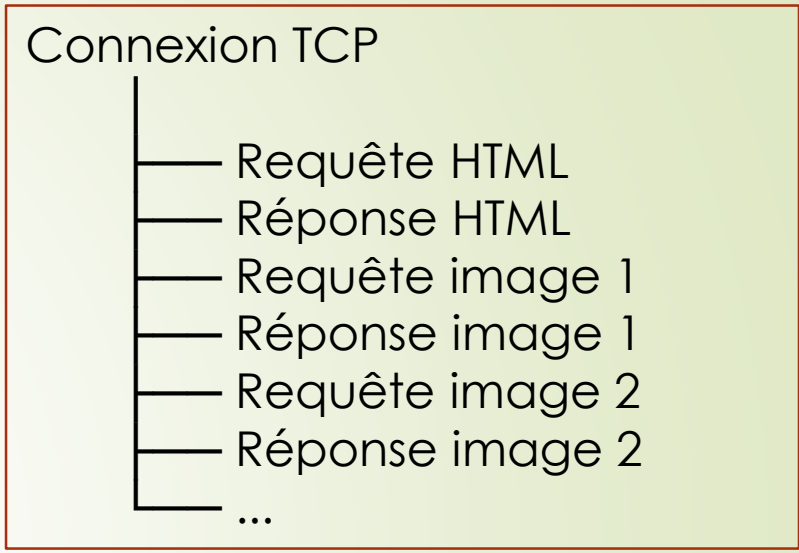
Evolution de HTTP

	HTTP/1.1	HTTP/2	HTTP/3
Format	texte	binaire	binaire
Transport	TCP	TCP	QUIC/UDP
Multiplexage	limité	Oui	Oui
Compression headers	Non	Oui	Oui
TLS	HTTPS séparé	généralement utilisé	intégré à QUIC

Notion de multiplexage dans HTTP

- Une page Web contient généralement plusieurs ressources :
  - document HTML
  - feuilles CSS
  - images
  - fichiers JavaScript
  - vidéos ou autres ressources
- Le navigateur doit donc effectuer plusieurs échanges avec le serveur.
- **Question :** comment transporter efficacement ces échanges ?
- → Le multiplexage permet de faire progresser plusieurs échanges simultanément sur une même connexion.

Multiplexage avec HTTP1



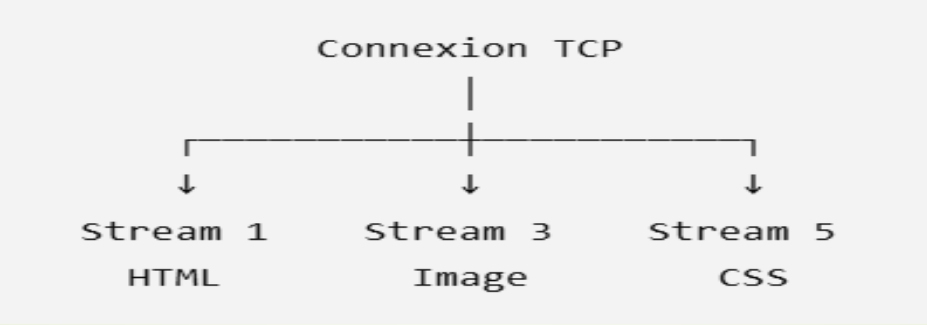
- Imaginons qu'une page Web contienne :
  - un fichier HTML
  - 3 images
  - un fichier CSS
  - 2 fichiers JavaScript
- Le navigateur doit donc récupérer plusieurs ressources.
- **Avec HTTP/1.1**
- Les requêtes sont généralement organisées sur plusieurs connexions TCP, ou sur une même connexion avec des requêtes successives selon les mécanismes utilisés.

Multiplexage avec HTTP/2: les streams

- HTTP/2 introduit la notion de **stream (flux)**.
- un stream représente un échange logique particulier.
- Chaque échange HTTP est associé à un stream identifié par un numéro.
- Exemple :

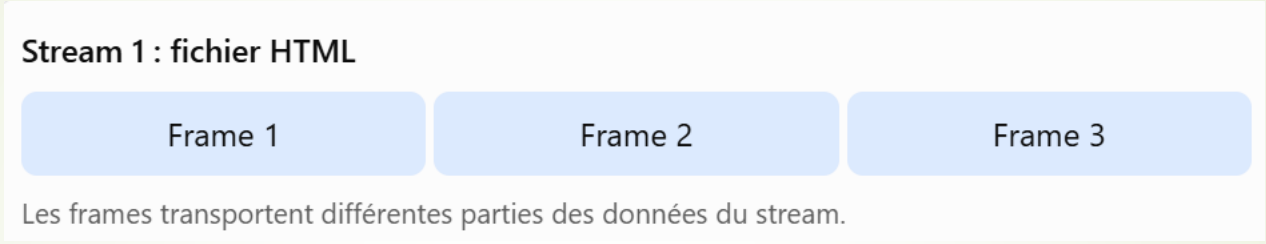
Stream 1 → HTML
Stream 3 → image.jpg
Stream 5 → style.css
Stream 7 → script.js

Tous ces streams utilisent **une seule connexion TCP**.

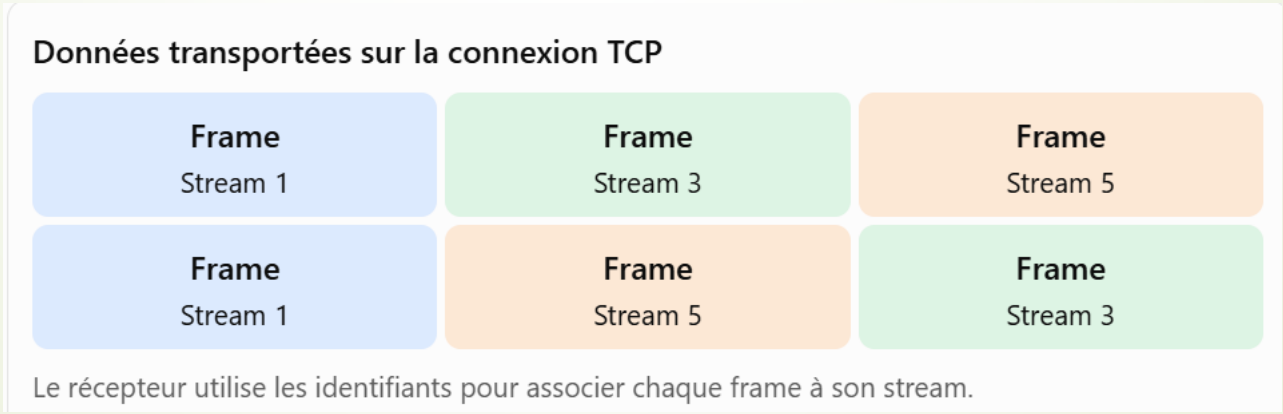


Multiplexage avec HTTP/2: les frames

- Les données d'un stream sont divisées en **frames**.
- Une frame (trame HTTP/2) est une petite unité de données utilisée pour transporter des informations dans un stream.



- Les frames des différents streams peuvent être **entrelacées** sur la même connexion.
- Exemple :



- Le récepteur utilise l'**identifiant du stream** pour savoir à quel échange appartient chaque frame.

Avantages et limites du Multiplexage avec HTTP/2

- **Avantages**

•plusieurs échanges progressent simultanément ;
•une seule connexion TCP peut être utilisée ;
•moins besoin de multiplier les connexions ;
•meilleure utilisation de la connexion.

- **Limite: TCP**

- TCP garantit que les données sont reçues **dans l'ordre**.
- Supposons :

•Stream 1 → paquet A → paquet B → paquet C
•Stream 3 → paquet D → paquet E

•Si un paquet TCP est perdu : paquet B par exemple

- TCP doit gérer la retransmission et maintenir son fonctionnement en ordre.
- Cela peut provoquer un **Head-of-Line Blocking (HOL)** au niveau du transport.

Multiplexage avec HTTP/3

- HTTP/3 utilise **QUIC** au lieu de TCP.

HTTP/3

↓

QUIC

↓

UDP

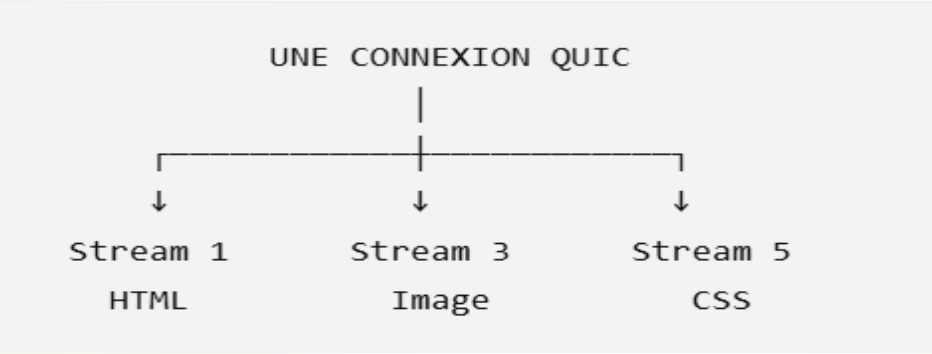
↓

IP

QUIC fournit notamment :
•des connexions ;
•plusieurs streams ;
•une gestion indépendante des streams ;
•une sécurité intégrée avec TLS 1.3.

Multiplexage avec HTTP/3

- Une connexion QUIC peut contenir plusieurs streams :



- Les streams sont gérés de manière **indépendante**.
- Si des données d'un stream sont perdues :

```
Stream 1 → perte ❌ → retransmission
Stream 3 → continue ✓
Stream 5 → continue ✓
```

- La perte sur un stream ne bloque donc pas les autres streams **au niveau du transport**.

HTTP/2 vs HTTP/3

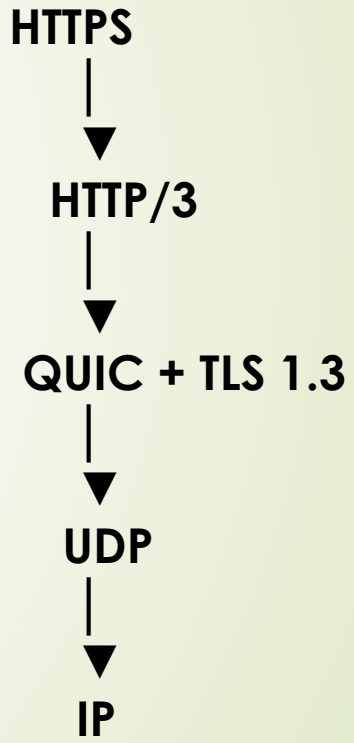
Caractéristique	HTTP/2	HTTP/3
Protocole de transport	TCP	QUIC
Fonctionnement sur UDP	✗	✓
Multiplexage	✓	✓
Plusieurs streams	✓	✓
Une seule connexion	✓	✓
Streams indépendants face aux pertes	✗ au niveau TCP	✓
Head-of-Line au niveau transport	Possible	Évité entre streams

- HTTP/2 bénéficie du multiplexage mais reste soumis aux contraintes de TCP, tandis que HTTP/3 utilise QUIC pour permettre une gestion indépendante des streams face aux pertes de paquets.

HTTPS (**HyperText Transfer Protocol Secure**) Techniquement **HTTP over TLS (Transport Layer Security)**

HTTPS is HTTP transmitted over a TLS-secured connection.





HTTPS is not a different application protocol from HTTP; it is HTTP protected by TLS.
Port: 443

Quelles sont les trois propriétés principales apportées par HTTPS ?

- **TLS (Transport Layer Security)** protège les échanges entre le client et le serveur.
- Il permet notamment de :
  - **chiffrer** les données échangées ;
  - garantir leur **intégrité** ;
  - **authentifier le serveur** grâce au certificat numérique.

Propriété	Signification
Confidentialité	Les données sont chiffrées et ne peuvent pas être facilement lues par un observateur du réseau.
Intégrité	Les données ne doivent pas être modifiées discrètement pendant leur transmission.
Authentification	Le certificat permet de vérifier l'identité du serveur auquel le client se connecte.

Synthèse

Le trajet d'une requête HTTPS

Du navigateur au serveur : quelles étapes ?

1 Saisie de l'URL

L'utilisateur saisit dans son navigateur :

https://www.example.com

2 Résolution DNS

Le navigateur interroge un serveur DNS pour obtenir l'adresse IP du domaine.

3 Établissement de la connexion

Le navigateur établit une connexion avec le serveur sur le port 443 (en utilisant QUIC pour HTTP/3 ou TCP pour HTTP/2).

4 TLS Handshake

Le client et le serveur négocient les paramètres de sécurité et établissent une clé de session. Le **serveur** présente son **certificat**.

5 Requête HTTP

Une fois la connexion sécurisée, le navigateur envoie la requête HTTP (ex. GET /index.html).

6 Traitement par le serveur

Le serveur traite la requête, accède aux ressources et prépare la réponse.

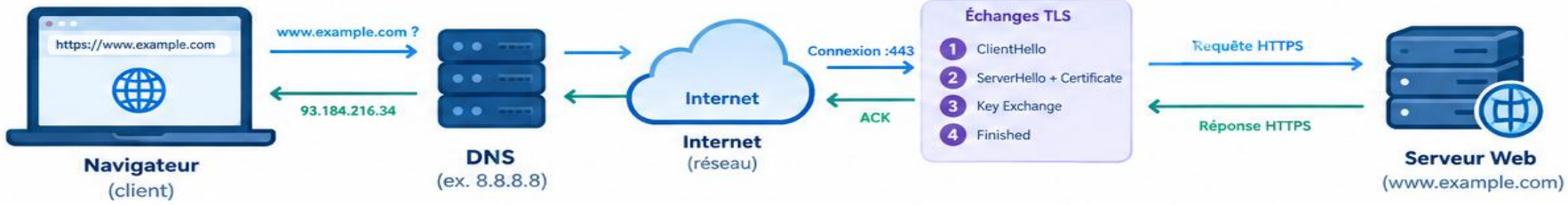
7 Réponse HTTP

Le serveur renvoie la réponse (HTML, JSON, etc.) dans le même canal chiffré.

8 Affichage du contenu

Le navigateur reçoit la réponse, déchiffre les données et rend la page Web.

Il peut ensuite charger d'autres ressources (CSS, JS, images...).



La pile de protocoles *(selon la version de HTTP)*

HTTP/1.1 *(HTTPS)*	HTTP/2 *(HTTPS)*	HTTP/3 *(HTTPS)*
HTTP/1.1	HTTP/2	HTTP/3
TLS	TLS	QUIC + TLS 1.3
TCP	TCP	UDP
IP	IP	IP

Rôle de HTTPS (HTTP over TLS)

- **Confidentialité** : les données sont chiffrées
- **Intégrité** : les modifications sont détectables
- **Authentification** : le certificat permet d'identifier le serveur

Ce qu'il faut retenir

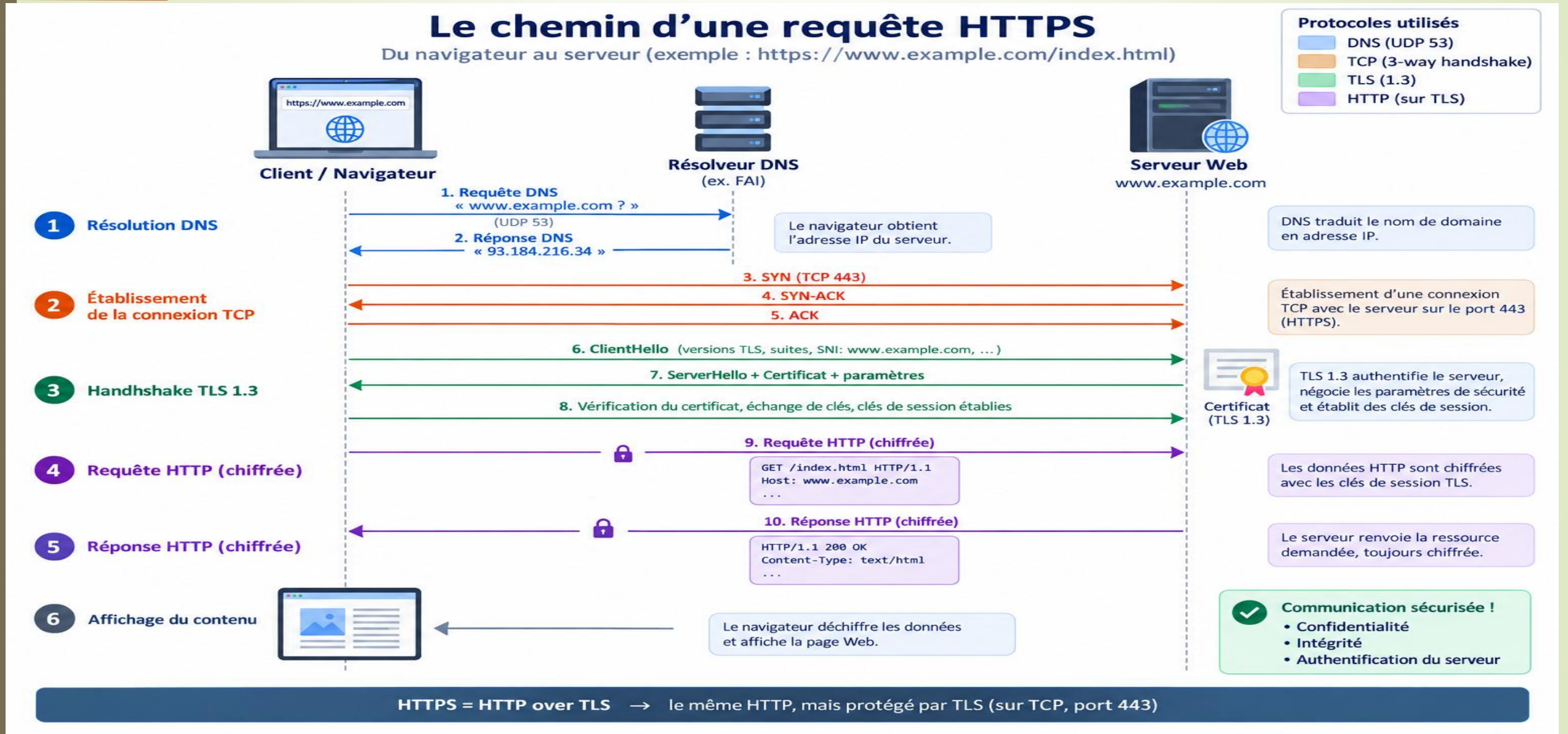
HTTPS n'est pas une nouvelle version de HTTP.
C'est HTTP protégé par TLS.

Le transport peut être différent :
- TCP (HTTP/1.1 et HTTP/2)
- QUIC/UDP (HTTP/3)

TLS 1.3 est la version moderne de TLS, plus rapide et plus sûre.

Le site est sécurisé, mais HTTPS ne garantit pas la fiabilité du contenu du site.

Synthèse


Le chemin d'une requête HTTPS
Du navigateur au serveur (exemple : https://www.example.com/index.html)
Protocoles utilisés
DNS (UDP 53)
TCP (3-way handshake)
TLS (1.3)
HTTP (sur TLS)
https://www.example.com
Client / Navigateur
Résolveur DNS
(ex. FAI)
Serveur Web
www.example.com
1. Requête DNS
« www.example.com ? »
(UDP 53)
2. Réponse DNS
« 93.184.216.34 »
1 Résolution DNS
Le navigateur obtient l'adresse IP du serveur.
DNS traduit le nom de domaine en adresse IP.
3. SYN (TCP 443)
4. SYN-ACK
5. ACK
2 Établissement de la connexion TCP
Établissement d'une connexion TCP avec le serveur sur le port 443 (HTTPS).
6. ClientHello (versions TLS, suites, SNI: www.example.com, ...)
7. ServerHello + Certificat + paramètres
8. Vérification du certificat, échange de clés, clés de session établies
3 Handhshake TLS 1.3
Certificat (TLS 1.3)
TLS 1.3 authentifie le serveur, négocie les paramètres de sécurité et établit des clés de session.
9. Requête HTTP (chiffrée)
4 Requête HTTP (chiffrée)
GET /index.html HTTP/1.1
Host: www.example.com
...
Les données HTTP sont chiffrées avec les clés de session TLS.
10. Réponse HTTP (chiffrée)
5 Réponse HTTP (chiffrée)
HTTP/1.1 200 OK
Content-Type: text/html
...
Le serveur renvoie la ressource demandée, toujours chiffrée.
6 Affichage du contenu
Le navigateur déchiffre les données et affiche la page Web.
Communication sécurisée !
• Confidentialité
• Intégrité
• Authentification du serveur
HTTPS = HTTP over TLS → le même HTTP, mais protégé par TLS (sur TCP, port 443)